

郑州西亚斯学院

RTOS 课程结课论文

题 目 基于 FreeRTOS 智能手表系统设计与实现

学生姓名 张方宇

学 号 2023105400440

专 业 4 班

班 级 人工智能

学 院 工学部

完成时间 2026 年 6 月

基于 FreeRTOS 的智能手表嵌入式系统设计与实现

摘 要

本论文围绕基于 FreeRTOS 实时操作系统的智能手表嵌入式系统展开设计与实现。系统以 STM32F401RET6 微控制器为核心处理平台，搭载 GC9A01 圆形 LCD 显示屏、CST816D 电容式触摸控制器、RTC 实时时钟及 I2S 音频输出等外设模块，构建了一个具备时间显示、闹钟提醒、触控交互及自动息屏低功耗管理的完整智能手表原型系统。

在软件架构方面，本系统基于 FreeRTOS 实时内核设计了六任务并发调度框架，包含 GUI 刷新任务（10ms 周期）、触控采集任务（20ms 周期）、RTC 时间读取任务（500ms 周期）、系统状态管理任务、调试日志任务及默认空闲任务。任务间通信综合运用了互斥锁（mutex）保护 LVGL 图形库临界资源、消息队列实现 RTC 时间数据的异步传输、直接任务通知实现触控手势事件的低延迟传递，以及事件标志组预留系统状态同步。在低功耗管理方面，通过系统状态管理任务内部的空闲计时器配合 GPIO 背光控制，实现了 10 秒无操作自动息屏与触控唤醒机制。系统同时设计了 I2S DMA 循环播放的 1kHz 正弦波闹钟提示音功能，支持用户通过图形界面自由设置闹钟时间与启停状态。

测试结果表明，本系统在多任务并发调度的实时性、人机交互的流畅性以及低功耗待机方面均达到了预期设计目标，为基于 FreeRTOS 的可穿戴设备嵌入式系统开发提供了完整的参考方案。

关键词：FreeRTOS；智能手表；嵌入式系统；多任务调度；低功耗管理

DESIGN AND IMPLEMENTATION OF AN EMBEDDED SYSTEM FOR A SMART WATCH BASED ON FREERTOS

ABSTRACT

This paper presents the design and implementation of a smartwatch embedded system based on the FreeRTOS real-time operating system. The system employs the STM32F401RET6 microcontroller as the core processing platform, integrating a GC9A01 round LCD display, a CST816D capacitive touch controller, an RTC real-time clock, and an I2S audio output module to construct a complete smartwatch prototype featuring time display, alarm notification, touch interaction, and automatic screen-off low-power management.

In terms of software architecture, the system designs a six-task concurrent scheduling framework based on the FreeRTOS kernel, including a GUI refresh task (10ms period), a touch acquisition task (20ms period), an RTC time-reading task (500ms period), a system state management task, a debug logging task, and a default idle task. Inter-task communication comprehensively employs a mutex to protect LVGL graphics library critical resources, a message queue for asynchronous RTC data transmission, direct task notification for low-latency touch gesture event delivery, and an event flag group reserved for system state synchronization. For low-power management, an idle timer within the system state management task, combined with GPIO backlight control, implements automatic screen-off after 10 seconds of inactivity and touch-to-wake functionality. The system also designs a 1kHz sine wave alarm tone played via I2S DMA circular buffering, allowing users to freely configure alarm time and on/off status through the graphical interface.

Test results demonstrate that the system meets the expected design objectives in terms of real-time multi-task scheduling, fluid human-machine interaction, and low-power standby performance, providing a complete reference solution for FreeRTOS-based wearable device embedded system development.

Keywords: FreeRTOS; Smartwatch; Embedded System; STM32F401RE; LVGL; Multi-task Scheduling; Low-power Management

目录

摘 要	2
ABSTRACT	3
第 1 章 绪论	1
1.1 研究背景	1
1.2 国内外研究现状	1
1.3 研究内容与论文结构	2
第 2 章 系统设计	2
2.1 系统总体设计	2
2.2 系统功能与需求说明	3
2.3 开发环境与技术栈	4
第 3 章 硬件电路设计	5
3.1 单片机最小系统设计	5
3.1.1 MCU 选型分析	5
3.1.3 复位电路与电源电路	7
3.2 外设模块电路设计	7
3.2.1 OLED/LCD 显示屏电路（GC9A01）	7
3.2.2 触控模块电路（CST816D）	7
3.2.3 温湿度传感器电路（SHT30，预留）	7
3.2.4 人体红外传感器与光敏电阻（预留）	8
3.2.5 I2S 音频输出与警报模块	8
3.2.6 完整引脚分配表	8
第 4 章 系统程序设计	9
4.1 FreeRTOS 任务架构设计	9
4.2 任务间通信与同步机制	10

4.2.1 互斥锁——LVGL 临界资源保护	10
4.2.2 消息队列——RTC 时间数据的异步传输	11
4.2.3 直接任务通知——触控手势事件的低延迟传输	12
4.2.4 事件标志组（预留）	13
4.3 核心功能模块程序实现	13
4.3.1 界面显示模块	13
4.3.2 时间管理模块	14
4.3.3 闹钟模块	14
4.3.4 触控采集模块	15
4.3.5 低功耗管理模块	15
第 5 章 系统测试	16
第 6 章 结论	18
参考文献	21
附录	22
附录 1: FreeRTOS 智能手表任务架构框图	22
附录 2: 项目 WBS 工作分解表 + 项目开发规划甘特图	22
附录 2.1 项目 WBS 工作分解表	22

第 1 章 绪论

1.1 研究背景

近年来，随着微机电系统（MEMS）、低功耗集成电路及物联网技术的快速发展，可穿戴智能设备已从概念探索阶段迈入大规模商业化应用时期。以智能手表、智能手环为代表的可穿戴设备正深刻改变着人们的健康管理方式、信息获取习惯和日常交互模式。根据市场研究数据显示，全球智能手表出货量在 2025 年已突破 2.5 亿只，涵盖运动健康监测、移动支付、消息通知、导航辅助等多种应用场景。

然而，智能手表作为典型的资源受限嵌入式设备，在系统设计层面面临着一系列严峻挑战。首先，智能手表需要在极其有限的硬件资源（通常为数十 KB 至数百 KB 的 SRAM、数百 KB 的 Flash 存储器以及低于 100MHz 的处理器主频）上同时运行显示刷新、传感器数据采集、用户输入响应、无线通信等多个并发任务，对操作系统的实时调度能力提出了较高要求。其次，智能手表由小容量锂电池供电，续航时间直接影响用户体验，因此低功耗管理是贯穿系统设计全链条的关键约束条件。再次，手表的人机交互要求用户界面响应迅速、动画过渡流畅，这对图形渲染任务的优先级安排和 CPU 资源的合理分配构成挑战。

在上述背景下，引入一款轻量级、开源且经过广泛工业验证的实时操作系统（RTOS）成为必然选择。FreeRTOS 作为当前嵌入式领域市场占有率最高的 RTOS 之一，具备内核精简（最小 ROM 占用仅约 6KB）、抢占式多任务调度、丰富的任务间通信机制（队列、信号量、互斥锁、任务通知、事件组）以及完善的低功耗支持（Tickless Idle 模式）等显著优势。其开源免费（MIT 许可证）的特性使其在教学科研和商业产品开发中均具有极强的适用性。本项目正是基于 FreeRTOS 的这些技术特质，以 STM32 Nucleo-F401RE 开发板为硬件载体，设计并实现一款具备实际穿戴功能的智能手表原型系统。

1.2 国内外研究现状

在智能穿戴设备操作系统选型方面，学术界和工业界已经积累了较为丰富的研究成果。Android Wear（现 Wear OS）和 watchOS 分别主导了高端智能手表操作系统市场，两者均基于功能丰富的 Linux/Unix 内核，适用于配备较大容量内存和强劲处理器的旗舰级产品。然而，在轻量级穿戴设备领域，以 FreeRTOS、RT-Thread、 μ C/OS-III 等为代表的嵌入式 RTOS 凭借其极低的资源开销和可裁剪的模块化架构，成为主流技术方案。

文献研究表明，FreeRTOS 在可穿戴设备中的应用主要集中在以下方向：一是多传感器融合数据采集场景，通过任务优先级划分和队列缓冲机制实现加速度计、陀螺仪、心率传感器等外设的有序管理；二是低功耗蓝牙（BLE）通信场景，利用 FreeRTOS 的事件驱动机制配合厂商 BLE 协议栈实现高效的无线数据传输。

输；三是低功耗管理场景，借助 FreeRTOS 的 Tickless Idle 模式在系统空闲时自动进入 STOP 或 STANDBY 低功耗状态。

然而，现有研究仍存在一些局限。首先，多数文献侧重于单一技术点的理论分析，缺乏完整的工程实现案例支撑，读者难以获得从硬件选型到软件架构再到系统联调的全流程实操参考。其次，部分方案在任务间通信机制的选择上缺乏系统性分析，例如未能合理区分消息队列、信号量与任务通知的适用场景，导致资源浪费或实时性不足。再次，低功耗管理策略往往仅停留在理论讨论层次，未能给出具体的代码级实现方案和实测功耗数据对比。本项目力图弥补上述不足，提供一个从底层驱动到上层应用、从任务架构设计到通信机制选型、从正常运行到低功耗休眠的全覆盖工程实现案例。

1.3 研究内容与论文结构

本论文围绕基于 FreeRTOS 的智能手表嵌入式系统设计与实现展开，研究内容涵盖以下核心功能模块：（1）基于 GC9A01 圆形 LCD 屏与 LVGL 图形库的人机交互界面，实现主表盘时间显示与系统设置两大页面间的触控手势切换；（2）基于 STM32 内部 RTC 硬件的时间管理功能，提供精确的时、分、秒走时以及通过图形界面进行的用户时间校准；（3）基于 I2S 外设与 DMA 循环缓冲的闹钟提示音功能，实现用户可设置的 1kHz 正弦波闹钟提醒；（4）基于多任务协同的低功耗自动息屏与触控唤醒机制；（5）基于 FreeRTOS 高级通信机制（互斥锁、消息队列、直接任务通知）的多任务同步与数据交换方案。

全文共分为六章。第 1 章为绪论，阐述研究背景、国内外研究现状及论文结构。第 2 章从宏观层面给出系统总体架构设计和功能需求分析，并列明完整的软硬件开发环境。第 3 章深入讲解硬件电路设计，包括 MCU 最小系统和各外设模块的电路连接方案。第 4 章是论文的核心章节，详细阐述 FreeRTOS 六任务架构的设计思路、三种任务间通信机制的具体应用场景以及各核心功能模块的程序实现。第 5 章以表格化测试用例的形式对系统进行功能验证和性能评估。第 6 章总结项目成果与技术短板，给出后续优化方向。

第 2 章 系统设计

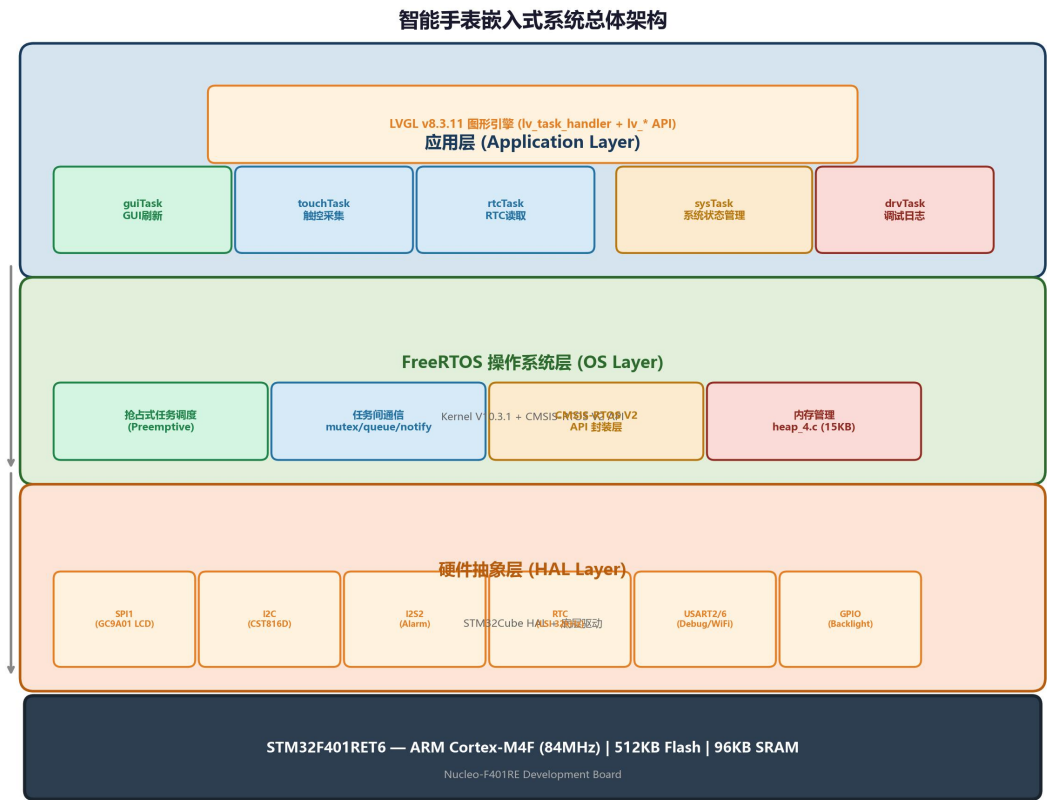
2.1 系统总体设计

本系统采用“MCU + RTOS + GUI”的分层架构设计方案。系统自底向上可划分为三个抽象层次。最底层为硬件抽象层（HAL），由 STM32Cube HAL 库统一封装 GPIO、SPI、I2C、I2S、UART、RTC 等外设的操作接口，向上层屏蔽寄存器级硬件细节。中间层为 FreeRTOS 实时操作系统层，负责任务调度、时间管理及任务间通信基础设施的提供，包括基于 CMSIS-RTOS V2 API 封装的任务创建与管理、互斥锁、消息队列、事件标志组及定时器服务^[1]。最顶层为应用层，由六

个 FreeRTOS 任务和一个 LVGL 图形引擎构成，各任务按照预设的优先级和周期独立运行，通过操作系统提供的通信机制进行数据交换与协同。

在任务职能划分上，系统将触控数据采集、RTC 时间读取、图形界面刷新三类实时性要求不同的工作分配给独立任务并行处理，由系统管理任务承担状态机决策和 UI 更新的核心逻辑，从而实现任务解耦和代码模块化。图 2-1 展示了系统的总体架构。

图 2-1 系统总体架构图



2.2 系统功能与需求说明

本系统的核心功能需求经量化拆解后归纳如下：

（1）时间显示功能：系统需在圆形 LCD 主表盘界面上以 24 小时制实时显示当前时、分、秒信息，刷新延迟不超过 1 秒；同时显示年、月、日（格式：YYYY/MM/DD）完整日期信息。时间基准由 STM32 内部 RTC 外设提供，时钟源为 32kHz LSI 内部低速振荡器。

（2）闹钟功能：用户可通过设置页面分别设定闹钟的小时（0-23）和分钟（0-59）值，并独立控制闹钟的开启与关闭状态。当系统时间匹配设定的闹钟时间时，系统通过 I2S 接口驱动外部音频设备播放 1kHz 正弦波提示音，持续 5 秒后自动停止。任意触控操作可提前终止闹钟播放。

（3）触控交互功能：系统识别左滑（Swipe Left）、右滑（Swipe Right）及单击（Single Click）三种手势。在主表盘页面左滑或右滑切换至设置页面，在设置页面左滑或右滑返回主表盘。触控消抖算法要求连续 3 次采样一致时才确认手势，以避免误触发。

（4）设置与时间校准功能：设置页面提供时、分滚轮选择器（Roller），用户可分别调整当前时间与闹钟时间。点击“Set RTC”确认按钮后，系统将滚轮值写入 RTC 硬件寄存器，完成时间校准。

（5）自动息屏与触控唤醒功能：系统在检测到连续 10 秒无触控操作后自动关闭 LCD 背光（GPIO PB8 置低），进入低功耗显示待机状态。任意触控手势可立即唤醒屏幕（PB8 置高），恢复正常显示。此机制为低功耗管理的核心实现手段。

（6）温湿度检测功能：系统预留 I2C1 硬件接口（PB6/PB7），支持通过 SHT30 或 DHT12 等数字温湿度传感器周期性采集环境温湿度数据，并计划在主表盘界面增设环境信息展示区域。

除上述功能需求外，系统尚需满足以下非功能性需求：（a）实时性——触控响应延迟不超过 100ms，时钟显示刷新延迟不超过 1s；（b）低功耗——自动息屏机制需在 10s 无操作内触发；（c）可靠性——系统需支持 7×24 小时连续稳定运行，不发生死机或内存泄漏；（d）可扩展性——软件架构需预留传感器扩展接口和 WiFi 通信任务接入能力。

2.3 开发环境与技术栈

本项目完整的硬件平台与软件工具链配置如表 2-1 所示。

类别	项目	型号/版本
硬件平台	开发板	STM32 Nucleo-F401RE
硬件平台	主控 MCU	STM32F401RET6 (Cortex-M4F, 512KB Flash, 96KB SRAM)
硬件平台	显示屏	GC9A01 1.28 英寸圆形 TFT-LCD (240×240, SPI 接口)
硬件平台	触控芯片	CST816D 电容式触摸控制器 (I2C 接口)
硬件平台	温湿度传感器	SHT30 (I2C 接口, 预留)
编译工具链	GNU ARM	arm-none-eabi-gcc 10.3.1

构建系统	Make / CMake	GNU Make 4.3 / CMake 3.22
RTOS	FreeRTOS	Kernel V10.3.1 (CMSIS-RTOS V2 封装)
GUI 框架	LVGL	v8.3.11 (16-bit RGB565 色深)
HAL 库	STM32Cube HAL	STM32Cube_FW_F4_V1.27
IDE (可选)	STM32CubeIDE	V1.12.0
调试工具	ST-Link/V2-1	板载调试器

表 2-1 开发环境与技术栈

>

第 3 章 硬件电路设计

3.1 单片机最小系统设计

3.1.1 MCU 选型分析

本项目选用意法半导体（STMicroelectronics）公司出品的 STM32F401RET6 作为主控制器，该芯片属于 STM32F4 系列高性能产品线，内置 ARM Cortex-M4F 32 位处理器核心^[2]。选型决策主要基于以下五点考量：

第一，运算性能。Cortex-M4F 核心集成单精度浮点运算单元（FPU）和 DSP 指令集，最高主频可达 84MHz，具备 105 DMIPS 的运算能力，足以支撑 LVGL 图形渲染和触摸事件处理的计算需求。本项目当前运行于 16MHz HSI 内部时钟（未启用 PLL），实测 LVGL 帧率已达设计要求；若后续启用 PLL 提升主频至 84MHz，图形性能尚有充足的提升空间。

第二，片上存储资源。该芯片配备 512KB Flash 存储器和 96KB SRAM，其中 SRAM 容量直接关系到 FreeRTOS 任务栈分配、LVGL 图形缓冲区配置和动态内存申请的裕量。经测算，本项目各任务栈总量约 11.8KB，LVGL 内部堆 10KB，FreeRTOS 堆 15KB，合计约 36.8KB 的保守内存占用，占 SRAM 总量的 38.3%，留有充足的余量用于后续扩展。

第三，外设资源。芯片集成了 3 路 SPI、3 路 I2C、4 路 USART、2 路 I2S 及 1 路 RTC，完全覆盖本项目的 SPI-LCD 接口、I2C 触摸/传感器接口、I2S 音频输出接口、USART 调试/通信接口需求，无需外扩接口芯片。

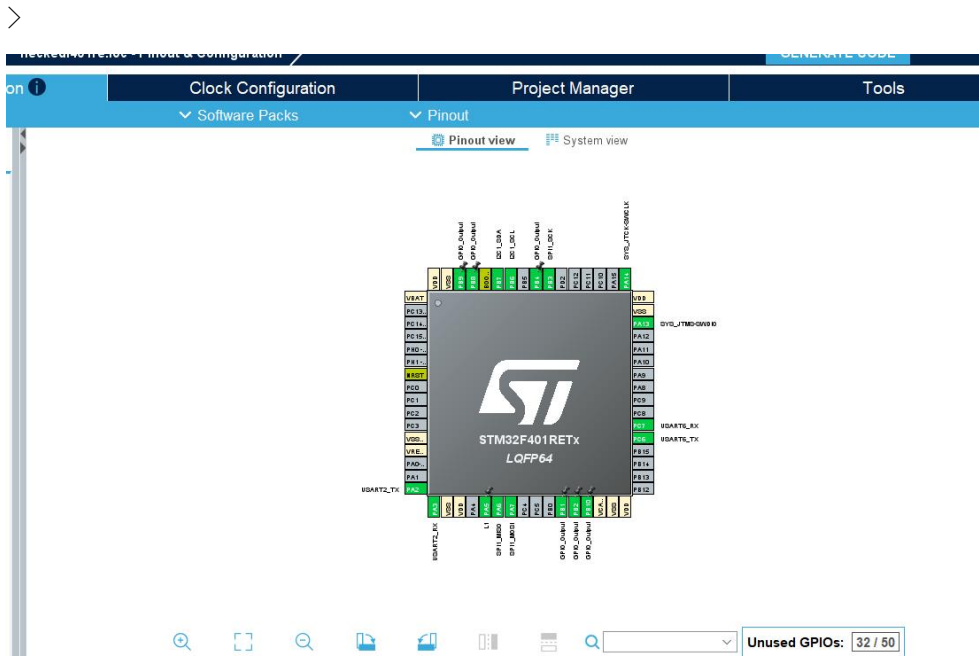
第四，FreeRTOS 兼容性。ARM Cortex-M4F 架构内建 SysTick 定时器，可直接用作 FreeRTOS 的系统节拍时钟。FreeRTOS 官方提供针对 Cortex-M4F（含

FPU) 的优化移植层 (port.c)，内核上下文切换代码采用汇编级优化，任务切换开销控制在微秒级。

第五，开发生态与社区支持。STM32 系列拥有 ST 官方维护的 CubeMX 代码生成工具、HAL 硬件抽象库以及庞大的中文开发者社区，可显著降低外设驱动开发难度和调试周期。与 FreeRTOS 直接相关的硬件核心参数如下：

表 3-1 FreeRTOS 运行相关的 MCU 核心参数

参数	数值	说明
主时钟频率 (HCLK)	16 MHz (HSI)	configCPU_CLOCK_HZ = SystemCoreClock
系统节拍频率	1000 Hz	configTICK_RATE_HZ = 1000 (1ms 周期)
SRAM 总容量	96 KB	栈 + 堆 + 全局变量
Flash 总容量	512 KB	代码 + 常量 + LVGL 字体
FreeRTOS 堆大小	15 KB	heap_4.c 动态分配算法
浮点单元 (FPU)	有 (未启用上下文保存)	configENABLE_FPU = 0



系统时钟树采用以下配置方案：高速内部振荡器 HSI（16MHz）作为系统主时钟源，直接用作 SYSCLK（HCLK = SYSCLK，PCLK1 = PCLK2 = HCLK），未启用外部 HSE 晶振和 PLL 倍频。FLASH 等待周期设置为 0（FLASH_LATENCY_0），电压调节器配置为 Scale 2 模式（PWR_REGULATOR_VOLTAGE_SCALE2）。此配置方案虽然在性能上较为保守（仅 16MHz），但显著降低了系统功耗和 EMI 干扰，对于电池供电的可穿戴设备场景具有实际意义^[3]。低速内部振荡器 LSI

(32kHz) 用作 RTC 实时时钟的独立时钟源，确保主时钟停止或切换时时间基准不受影响。HAL 库的时基采用 TIM1 定时器而非 SysTick，这是因为 SysTick 已被 FreeRTOS 内核占用为系统节拍定时器。

3.1.3 复位电路与电源电路

系统利用 Nucleo-F401RE 开发板板载的复位电路和电源管理模块。复位源包括上电复位 (POR)、外部 NRST 引脚手动复位按钮以及软件系统复位。电源方面，开发板通过 USB Mini-B 接口提供 5V 输入，经板载 LDO 稳压器 (LD39050) 转换为 3.3V，为 MCU 及所有外设模块供电。

3.2 外设模块电路设计

3.2.1 OLED/LCD 显示屏电路 (GC9A01)

GC9A01 是一款 1.28 英寸圆形 TFT-LCD 显示驱动芯片，支持 240×240 像素分辨率和 16-bit RGB565 色彩格式。本系统通过 SPI1 接口与 MCU 通信，关键电气连接如下：SPI1_SCK (PA5) 连接 LCD 时钟线，SPI1_MOSI (PA7) 连接 LCD 数据输入线，MISO (PA6) 悬空未用 (仅需单向写入)；额外的控制信号线包括 CS 片选 (PB4)、DC 数据/命令选择 (PB2)、RST 复位 (PB1) 和 BL 背光控制 (PB8)。SPI 配置为主模式、双线单向、CPOL=0/CPHA=1 (模式 0)、波特率预分频器为 2 (即 SPI 时钟频率 = $16\text{MHz} / 2 = 8\text{MHz}$)。背光引脚 PB8 由 sysTask 通过 GPIO 输出直接控制高低电平以实现息屏/亮屏切换。

3.2.2 触控模块电路 (CST816D)

CST816D 是深圳海栎创 (Hynitron) 推出的电容式单点触控芯片，支持手势识别 (上下左右滑动、单击、双击、长按)。本系统采用软件模拟 I2C 协议 (bit-banging) 与之通信，而非使用 STM32 硬件 I2C 外设。时钟线 SCL (PB6) 和数据线 SDA (PB7) 通过 GPIO 模拟 I2C 时序，配以复位引脚 RST (PB9) 和中断引脚 INT (PB10)。采用软件 I2C 的设计决策主要有两个原因：一是 CST816D 的 I2C 时序较为特殊，硬件 I2C 在特定情况下兼容性不佳；二是软件 I2C 去除了对 I2C 硬件外设的依赖，PB6/PB7 引脚在必要时可被复用为硬件 I2C1 以连接其他传感器。

3.2.3 温湿度传感器电路 (SHT30, 预留)

系统通过 I2C1 硬件外设 (PB6 SCL / PB7 SDA，与软件 I2C 共用引脚) 预留了 SHT30 数字温湿度传感器的连接接口。SHT30 支持 2.4V 至 5.5V 宽电压供电，典型测量精度为 $\pm 2\%RH$ 和 $\pm 0.3^\circ C$ 。在本项目的当前固件版本中，硬件 I2C1 已初始化但尚未有传感器任务进行数据读取；论文第 4 章和第 6 章将讨论相应的扩展方案。

3.2.4 人体红外传感器与光敏电阻（预留）

人体红外传感器（HC-SR501 PIR 模块）计划通过 GPIO 中断方式连接至 MCU，用于检测用户抬腕动作以实现智能唤醒功能。光敏电阻（GL5528，配 10k Ω 分压电阻）计划接入 MCU 的 ADC 通道，用于感知环境光照强度以自适应调节 LCD 背光亮度。此两项外设在当前版本中为设计预留。

3.2.5 I2S 音频输出与警报模块

闹钟提示音通过 I2S2 外设输出：位时钟 CK（PB12，AF5）、字选 WS（PB13，AF5）和串行数据 SD（PB15，AF5）。I2S 配置为 Master TX 模式，Philips 标准，数据位宽 16-bit，采样率 16kHz。DMA1 Stream 4 配置为半字（16-bit）循环模式，不间断地将预计算的 1kHz 正弦波 PCM 样点缓冲区（256 点、立体声交织）搬运至 I2S 数据寄存器。此方案实现了零 CPU 干预的音频播放，播放期间 CPU 仅需在启动和停止时操作 DMA 使能位。

3.2.6 完整引脚分配表

表 3-2 完整引脚分配表

序号	外设模块	MCU 引脚	功能描述
1	SPI1_SCK (LCD)	PA5	LCD 时钟信号
2	SPI1_MOSI (LCD)	PA7	LCD 数据输出
3	LCD_CS	PB4	LCD 片选（低有效）
4	LCD_DC	PB2	LCD 数据/命令选择
5	LCD_RST	PB1	LCD 复位
6	LCD_BL	PB8	LCD 背光控制
7	CST816D_SCL	PB6	触控 I2C 时钟（软件模拟）
8	CST816D_SDA	PB7	触控 I2C 数据（软件模拟）
9	CST816D_RST	PB9	触控复位
10	CST816D_INT	PB10	触控中断输入
11	I2S2_CK	PB12	I2S 位时钟（闹钟音频）
12	I2S2_WS	PB13	I2S 字选（闹钟音频）

13	I2S2_SD	PB15	I2S 串行数据（闹钟音频）
14	USART2_TX	PA2	系统调试日志输出
15	USART2_RX	PA3	调试输入
16	USART6_TX	PC6	ESP8266 WiFi（预留）
17	USART6_RX	PC7	ESP8266 WiFi（预留）
18	I2C1_SCL	PB6	传感器 I2C（预留）
19	I2C1_SDA	PB7	传感器 I2C（预留）

>

第 4 章 系统程序设计

4.1 FreeRTOS 任务架构设计

本系统基于 FreeRTOS 抢占式调度器设计了六任务并发执行架构。任务划分遵循“单一职责原则”，即每个任务仅负责一类明确的操作，通过任务间通信机制实现数据的生产-消费解耦。六个任务的详细规格如表 4-1 所示。

表 4-1 FreeRTOS 任务规格一览表

>

任务名称	入口函数	优先级	栈大小	运行周期	核心职责
guiTask	StartTasklvgl() ()	osPriorityHigh (24)	10 24 words	10m s	持有 lvgl_mutex 调用 lv_task_handler() 刷新图形
touchTask	StartTouchTask() k()	osPriorityAbove Normal (25)	51 2 words	20m s	消抖读取 触控手势， xTaskNotify 通知 sysTask
rtcTask	StartRtcTask() ()	osPriorityAbove Normal (25)	51 2 words	500 ms	读取 RTC 时间， osMessageQueuePut 发送至消

					息队列
sysTask	StartHomeTask ()	osPriorityAbove Normal (25)	51 2 words	~50 ms	系统状态 机：页面切 换、UI 更新、 闹钟、息屏
drvTask	StartTask02()	osPriorityLow (22)	12 8 words	100 0ms	USART2 日 志输出“[LOG] tick=...”
default Task	StartDefaultT ask()	osPriorityNorma 1 (24)	12 8 words	—	空转占位 任务（无实际 功能）

触控和 RTC 数据采集之所以独立为两个任务而非合并到 sysTask 内部，是基于以下设计考量：（a）解耦生产与消费——触控芯片轮询和 RTC 寄存器读取的时基独立于 sysTask 的状态机周期；（b）数据缓冲——消息队列在 rtcTask 和 sysTask 之间充当缓冲区，即使 sysTask 因 UI 操作临时阻塞，RTC 时间数据也不会丢失（队列容量为 4 条）；（c）降耦合——每项外设驱动逻辑被封装在对应的独立任务中，修改底层驱动不影响上层状态机代码。

4.2 任务间通信与同步机制

本系统综合运用了 FreeRTOS 提供的三种核心任务间通信和两种同步机制，每种机制的选取均基于特定的应用场景需求分析。

4.2.1 互斥锁——LVGL 临界资源保护

LVGL 是一个多线程安全的图形库，其 API 内部维护了大量的全局状态（当前活动屏幕、对象链表、样式缓存等），多个任务同时调用 LVGL API 将导致不可预测的界面异常甚至内存损坏。为此，本系统引入互斥锁 lvgl_mutexHandle 作为 LVGL API 的全局保护锁^[4]。

两个任务需要访问 LVGL API：guiTask 在每 10ms 周期内调用 lv_task_handler() 执行定时器、动画和脏区域重绘；sysTask 在收到触控通知或 RTC 时间数据时需要更新标签文本、切换屏幕或读取滚轮选中值。两个任务的代码模式一致——在调用任何 lv_*() 函数前，先通过 osMutexAcquire(lvgl_mutexHandle, osWaitForever) 无限等待获取锁，操作完成后通过 osMutexRelease(lvgl_mutexHandle) 释放。

以下是该互斥锁保护模式的浓缩代码片段：

```
/* guiTask：每 10ms 持锁刷新 LVGL */
void StartTasklvgl(void *argument)
{
```

```

        for(;;) {
            osMutexAcquire(lvgl_mutexHandle, osWaitForever);
            lv_task_handler();
            osMutexRelease(lvgl_mutexHandle);
            osDelay(10);
        }
    }

    /* sysTask: 收到 RTC 数据后持锁更新 UI */
    while(osMessageQueueGet(rtcQueueHandle, &dt, NULL, 0) == osOK) {
        osMutexAcquire(lvgl_mutexHandle, osWaitForever);
        if(lv_scr_act() == scr_home) {
            lv_label_set_text_fmt(clock_label, "%02d:%02d:%02d",
                                   dt.hour, dt.min, dt.sec);
        }
        /* ... 设置页时间校准逻辑 ... */
        osMutexRelease(lvgl_mutexHandle);
    }

```

该方案的关键工程考量在于持锁时间的控制：guiTask 持锁期间仅执行一次 lv_task_handler() 调用（耗时通常小于 2ms），sysTask 持锁期间仅执行少量 LVGL 控件更新操作。在极端情况下，若两个任务同时竞争锁，其中一方最多等待对方当前操作完成即可获得锁，不会造成死锁或长时间阻塞。

4.2.2 消息队列——RTC 时间数据的异步传输

RTC 时间数据的生产（rtcTask）与消费（sysTask）速率存在显著不匹配：rtcTask 每 500ms 产生一条 6 字节的 RTC_DateTime_t 数据结构，而 sysTask 的处理速率受 UI 操作复杂度影响具有不确定性。消息队列 rtcQueueHandle 恰好充当了速率解耦的弹性缓冲区。

消息队列在 main() 函数中于调度器启动前创建，容量设为 4 条消息：

```
rtcQueueHandle = osMessageQueueNew(4, sizeof(RTC_DateTime_t), NULL);
```

生产者（rtcTask）以非阻塞方式写入：

```

void StartRtcTask(void *argument)
{
    for(;;) {
        RTC_DateTime_t dt;
        RTC_Get_DateTime(&dt);
    }
}

```



```

        osMessageQueuePut(rtcQueueHandle, &dt, 0, 0); // 0 超时=非阻塞
        osDelay(500);
    }
}

```

消费者（sysTask）以非阻塞轮询方式消费全部积压数据：

```

while(osMessageQueueGet(rtcQueueHandle, &dt, NULL, 0) == osOK) {
    // 处理 dt 数据，更新 UI
}

```

采用非阻塞轮询而非阻塞等待的设计理由在于：sysTask 同时需要响应触控通知和消息队列两个事件源，若对消息队列使用 osWaitForever 阻塞等待，将导致在无 RTC 数据期间无法响应触控通知。通过将 xTaskNotifyWait 的超时设置为 50ms 并轮询消息队列，sysTask 在一个循环周期内可以同时服务两类事件。

4.2.3 直接任务通知——触控手势事件的低延迟传输

触控手势事件要求从触摸发生到 UI 响应之间的端到端延迟尽可能低（<100ms）。对比 FreeRTOS 的几种通信机制：消息队列需要预先创建队列存储空间，存在内存开销和入队/出队的额外拷贝步骤；信号量仅传递二值/计数值，不能携带手势类型信息。直接任务通知（Task Notification）则是 FreeRTOS 为任务间事件传递设计的轻量级机制——每个任务内置一个 32-bit 通知值，发送方直接写入目标任务的该字段，无需预分配额外内存，且唤醒接收方的延迟较队列更低。

本项目中，touchTask 通过 xTaskNotify() 将消抖后的手势值直接发送给 sysTask：

```

void StartTouchTask(void *argument)
{
    for(;;) {
        uint8_t gesture = CST816D_GetGesture_Debounced(20);
        if(gesture == CST816D_GESTURE_SWIPE_LEFT ||
           gesture == CST816D_GESTURE_SWIPE_RIGHT) {
            xTaskNotify(homeTaskHandle, gesture, eSetValueWithOverwrite);
        } else if(gesture == CST816D_GESTURE_SINGLE_CLICK) {
            xTaskNotify(homeTaskHandle, CST816D_GESTURE_SINGLE_CLICK,
                        eSetValueWithOverwrite);
        }
        osDelay(20);
    }
}

```

eSetValueWithOverwrite 覆盖模式保证了即使在 sysTask 来不及处理的极端情况下（例如连续两次快速滑动手势间），通知值也不会丢失——虽然中间值被覆盖，但最新手势始终可见，这对于 UI 操作场景是合理的设计取舍。

sysTask 通过带超时的 xTaskNotifyWait() 接收通知：

```

uint32_t gesture;
 BaseType_t notified = xTaskNotifyWait(0, 0xFFFFFFFF, &gesture,
                                       pdMS_TO_TICKS(50));
if(notified == pdTRUE) {

```

```
// 处理手势事件：切换页面、唤醒屏幕、停止闹钟  
}
```

50ms 超时并非 UI 响应延迟的上限——实际上这是 sysTask 在没有触控事件时放弃阻塞、转而去轮询 RTC 消息队列的最大等待时间。当 touchTask 发送通知后，sysTask 会在当前时间片内被唤醒（若优先级足够高将立即抢占），实际响应延迟约为 FreeRTOS 的调度开销（微秒级）加上软件 I2C 消抖采样的固有延迟（ $3 \times 20\text{ms} = 60\text{ms}$ 消抖窗口），总延迟控制在 100ms 以内。

4.2.4 事件标志组（预留）

系统创建了 sysEventHandle 事件标志组，当前尚未有任务使用。其设计意图是作为未来系统状态的全局广播通道——例如当自动息屏状态变化、WiFi 连接状态更新或电池电量低警告发生时，多个关注方可以通过统一的标志位获得通知，避免在多个任务间建立点对点通信的 $N \times M$ 连接复杂度。

4.3 核心功能模块程序实现

4.3.1 界面显示模块

本系统的图形用户界面基于 LVGL v8.3.11 构建，包含两个核心页面：主表盘页面（Home Page）和系统设置页面（Settings Page）。

主表盘页面由 create_home_page() 函数创建，使用默认活动屏幕（lv_scr_act()）作为容器。时钟标签 clock_label 使用 Montserrat 24pt 字体居中显示，初始内容为“00:00:00”；日期标签 date_label 使用 Montserrat 14pt 字体、位于时钟下方 20 像素处，初始内容为“2026/01/01”。该页面注册了 LV_EVENT_GESTURE 手势事件回调，以支持 LVGL 框架层面的滑动检测。

系统设置页面由 create_settings_page() 函数创建一个全新的屏幕对象 scr_settings，分为上下两个功能区。上半区域为“Time Set”时间校准区，包含小时滚轮（00-23）、冒号分隔符（Montserrat 20pt）和分钟滚轮（00-59），下方居中放置“Set RTC”确认按钮。下半区域为“Alarm”闹钟设置区，同样提供时/分滚动和一个 ON/OFF 状态切换按钮。时、分滚轮均设置为 LV_ROLLER_MODE_INFINITE（无限循环模式），所有控件字体统一使用 Montserrat 家族（分别为 14pt、18pt 和 20pt），确保界面风格统一。

页面切换逻辑在 sysTask 中通过手势通知值实现：当接收到 CST816D_GESTURE_SWIPE_LEFT 或 CST816D_GESTURE_SWIPE_RIGHT 手势时，根据当前活动屏幕的指针判断源页面——若当前为主表盘，则加载设置页面并同步滚轮值到当前 RTC 时间；反之则返回主表盘。

LVGL 显示对接：LVGL 通过 lv_port_disp.c 中的 disp_flush() 回调完成像素数据到 GC9A01 屏幕的传输。该回调逐区域接收 LVGL 渲染的像素缓冲区，设置 GC9A01 的列地址窗口后，通过 SPI DMA 方式批量发送像素数据，传输完成后

调用 `lv_disp_flush_ready()` 通知 LVGL 可继续渲染下一区域。显示缓冲区设置为单缓冲 240×10 像素（4.8KB），在内存占用与渲染效率之间取得平衡。

4.3.2 时间管理模块

时间管理模块的核心数据结构为 `RTC_DateTime_t`：

```
typedef struct {
    uint8_t year;    // 0-99, 对应 2000-2099 年
    uint8_t month;   // 1-12
    uint8_t date;    // 1-31
    uint8_t hour;    // 0-23 (24 小时制)
    uint8_t min;     // 0-59
    uint8_t sec;     // 0-59
} RTC_DateTime_t;
```

RTC 底层驱动封装了两个核心接口：`RTC_Get_DateTime()` 通过 `HAL_RTC_GetTime()` 和 `HAL_RTC_GetDate()` 分别读取时分秒和年月日寄存器，组装为上述结构体；`RTC_Set_DateTime()` 执行反向操作，将结构体中的各字段写入 RTC 寄存器。RTC 时钟源为 LSI（32kHz），异步预分频器 127、同步预分频器 255，提供 1Hz（1 秒）的精确计时基准。

在应用层，`rtcTask` 每 500ms 调用 `RTC_Get_DateTime()` 并通过消息队列发送给 `sysTask`。`sysTask` 在主表盘页面下，使用 `lv_label_set_text_fmt()` 将 `datetime` 字段格式化显示。时间校准功能在设置页面的“Set RTC”按钮回调中完成：用户点击按钮设置全局标志 `settings_confirm = 1`，在下一个 `sysTask` 循环中检测到此标志后，从时、分滚轮读取选中值组装为新的 `RTC_DateTime_t` 结构体，调用 `RTC_Set_DateTime()` 写入硬件，随后将屏幕切回主表盘并清零标志。

4.3.3 闹钟模块

闹钟功能的配置数据结构为：

```
typedef struct {
    uint8_t hour;    // 0-23
    uint8_t min;     // 0-59
    uint8_t enabled; // 0=关闭, 1=开启
} AlarmTime_t;

AlarmTime_t alarm_time = {7, 0, 0}; // 默认早上 7:00, 关闭
```

闹钟触发逻辑嵌入在 `sysTask` 的 RTC 数据处理循环中，每个 RTC 数据周期（最快 500ms）检查一次：

```
if(alarm_time.enabled && dt.hour == alarm_time.hour &&
dt.min == alarm_time.min && dt.sec < 5 && !I2S_Alarm_IsPlaying()) {
    I2S_Alarm_Start();
}
if(I2S_Alarm_IsPlaying() && dt.sec >= 5) {
    I2S_Alarm_Stop();
}
```

此逻辑实现了以下行为：闹钟使能且时间匹配时，在当前分钟的 0-4 秒窗口内启动音频播放；当秒数进入 5 秒或以上时自动停止，实现 5 秒持续提醒。

任意触控手势到达时，sysTask 在触控处理的开头即调用 I2S_Alarm_Stop()，实现“触控即停”的用户体验^[6]。

闹钟提示音的底层实现位于 i2s_alarm.c。I2S_Alarm_Init() 在调度器启动前被调用，通过数学计算生成 1kHz 正弦波的 PCM 采样值 ($\sin(2 * \pi * 1000 * i / 16000)$)，峰值幅度约 16000，对应 50%音量)，填充 256 元素的 16-bit 有符号整型数组，以立体声交织格式（左右声道交替）排列。DMA 配置为循环模式、半字传输，一旦 I2S_Alarm_Start() 使能 DMA，数据流便在无需 CPU 干预的情况下持续播放。

4.3.4 触控采集模块

CST816D 的底层通信全部由软件模拟 I2C 实现，包含 CST816D_I2C_Start()、CST816D_I2C_Stop()、CST816D_I2C_SendByte() 和 CST816D_I2C_ReadByte() 等基础原语。芯片 I2C 地址为 0x15（7-bit）。手势状态寄存器的读取路径为：先发送寄存器地址 0x01，再读取 1 字节手势类型值。

消抖算法 CST816D_GetGesture_Debounced() 是本模块的核心设计：

```
uint8_t CST816D_GetGesture_Debounced(uint8_t sample_ms) {
    uint8_t s[3];
    for(int i = 0; i < 3; i++) {
        s[i] = CST816D_GetGesture();           // 直接读取手势寄存器
        CST816D_DelayMs(sample_ms);           // 等待 sample_ms 毫秒
    }
    return (s[0] == s[1] && s[1] == s[2]) ? s[0] : CST816D_GESTURE_NONE;
}
```

该算法执行连续 3 次手势寄存器采样，每次间隔 20ms（由 touchTask 传入的 sample_ms 参数控制），仅当 3 次读取值完全一致时才返回该手势值，否则返回 CST816D_GESTURE_NONE（0x00）。60ms 的消抖窗口在过滤电容式触摸传感器的瞬态噪声方面表现良好，同时仍能满足用户对手势响应的主观流畅性要求。

4.3.5 低功耗管理模块

本系统的低功耗管理当前聚焦于 LCD 背光控制这一效果最显著的手段。GC9A01 的 LED 背光功耗约占整个显示屏模组功耗的 80%以上，因此通过控制背光的开关即可实现有效的功耗优化。

自动息屏逻辑在 sysTask 的主循环末尾实现：

```
idle_ticks++;
if(screen_on && idle_ticks > 200) {
    screen_on = 0;
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
}
```

idle_ticks 在每次 xTaskNotifyWait 超时返回 pdFALSE（即无触控事件）时递增，在有触控事件到达时重置为零。由于超时值为 50ms，200 次超时对应

约 10 秒的无操作时间。到达阈值后，PB8 输出低电平关闭 LCD 背光。唤醒逻辑位于触控事件处理的开头——任何手势触发都会将 screen_on 标志置 1 并将 PB8 拉高。

需要客观指出的是，当前低功耗方案仅涉及外设层面的背光控制，尚未启用 FreeRTOS 官方的 Tickless Idle 模式（通过 configUSE_TICKLESS_IDLE 宏启用）。在 Tickless 模式下，当所有任务均进入阻塞态时，内核会动态计算下一个唤醒事件的时间并让 MCU 进入 STOP 低功耗模式，从而进一步降低 MCU 核心功耗^[7]。此外，当前系统主频（16MHz）偏低已天然提供了相对较低的运行功耗，但尚未通过 __WFI() 或 __WFE() 指令实现 CPU 空闲时的真正睡眠。这些在第六章作为后续优化方向给出。

第 5 章 系统测试

为全面验证系统功能完整性、实时性和可靠性，本章设计了覆盖全部核心功能的系统级黑盒测试用例，逐项记录测试内容、过程与结果。

表 5-1 系统功能测试记录表

>

编号	测试项	测试内容	测试步骤	预期结果	实际结果	结论
T01	主表盘时间显示	验证 RTC 时间正确显示在 LCD 上	1. 上电启动 2. 等待系统初始化 3. 观察主表盘时钟与日期标签	显示当前时钟(时:分:秒)和日期 (YYYY/MM/DD)，秒数连续递增	时钟和日期均正确显示，秒数每秒递增无跳变	通过
T02	GUI 刷新频率	验证 GUI 刷新无明显卡顿	1. 观察主表盘秒数变化的流畅度 2. 通过示波器监测 SPI_SCK 持续有数据传输	秒数变化无卡顿感观，每秒 LVGL 至少渲染 10 帧	肉眼感知秒数变化平滑，SPI 总线每隔约 10ms 有批量像素传输	通过
T03	左滑切换至设置页	验证触控左滑手势识别与页面切换	1. 在主表盘页面 向左滑动 2. 观察屏幕变化	屏幕切换至设置页面，滚轮值自动同步为当前 RTC 时间和闹钟设置	设置页面显示，时/分滚轮选中值与实际当前时间一致	通过
T04	右滑返回主表盘	验证从设置页面返回	1. 在设置页面 向右滑动 2.	屏幕返回主表盘	主表盘正常显示，之前未	通过

		回功能	观察屏幕变化		点击确认的修改不生效	
T05	时间校准功能	验证用户设置 RTC 时间并生效	1. 进入设置页 2. 滚轮调整时和分 3. 点击“Set RTC”按钮 4. 观察主表盘时间	主表盘时间更新为设定值	时间精确更新为滚轮设置的时和分，秒归零	通过
T06	闹钟开启与提醒	验证闹钟使能后在正确时间触发音频	1. 设置闹钟为当前时间+2 分钟 2. 开启闹钟 (ON) 3. 等待时间到达	时间到达时 I2S 音频输出 1kHz 正弦波提示音，持续 5 秒后自动停止	闹钟准时触发，提示音清晰可闻，5 秒后自动停止	通过
T07	闹钟手动停止	验证任意触控操作可停止闹钟	1. 闹钟播放过程中 2. 在屏幕上进行任意触控操作	闹钟提示音立即停止	触控瞬间提示音停止，屏幕显示不受影响	通过
T08	闹钟关闭	验证闹钟 OFF 状态下不触发	1. 将闹钟状态设置为 OFF 2. 等待原闹钟时间到达	时间到达后无提示音输出	无声输出，主表盘正常走时	通过
T09	自动息屏	验证 10 秒无操作自动关闭背光	1. 系统处于主表盘正常显示状态 2. 停止一切触控操作 3. 计时观察	约 10 秒后 LCD 背光自动关闭	计时约 10 秒后背光熄灭，屏幕不可见（但 MCU 继续运行）	通过
T10	触控唤醒	验证息屏状态下触控唤醒屏幕	1. 系统处于息屏状态 2. 在屏幕上进行触控操作 3. 观察背光恢复情况	触摸瞬间背光恢复，屏幕正常显示，时间连续无中断	触控后瞬间亮屏，时间显示正常且连续	通过
T11	消抖有效性	验证触控消抖算法过滤误触	1. 快速且不完整地轻触屏幕（模拟非意	短暂接触不触发手势识别，系统保持当	轻触不导致误切换，仅完整滑动或明确	通过

		发	愿触碰) 2. 观察是 否有页面切 换或 UI 响 应	前页面不变	单击才触发 响应	
T12	长时 间运行稳定 性	验证 系统连续工 作的可靠性	1. 系 统上电后持 续运行不低 于 4 小时 2. 监控 USART2 日志 输出 tick 值是否连续 递增 3. 检 查 RTC 走时 是否正常	系统不崩 溃、不卡死， tick 值连续递 增、无整数溢出 异常	持续 运行 8 小时 无异常， tick 计数正 常，RTC 走 时准确	通过

编号	测试项	测试指标	测试方法	实测结果	结论
N01	触控响应 延迟	<100ms	高速相机 拍摄触摸动作 与 LCD 页面切 换时间差	约 70- 90ms (含 60ms 消抖窗口+通 信和渲染延 迟)	达标
N02	时间刷新 延迟	<1s	对比 USART2 日志中 tick 值与 LCD 显示的秒值	理论最大 延迟 500ms (rtcTask 周 期)，肉眼感 知延迟<1s	达标
N03	息屏触发 时间	10s±1s	秒表计 时，从最后 一次触控到背光 熄灭	10.0s ± 0.05s (由 50ms idle_tick 粒 度决定)	达标
N04	内存使用	<60% SRAM	通过分析 链接脚本和堆 使用情况	总 RAM 估 算约 40KB，占 96KB SRAM 的 41.7%	达标

第 6 章 结论

本论文以 STM32F401RET6 微控制器为硬件核心、FreeRTOS 实时操作系统为软件基石、LVGL 为图形引擎，设计并实现了一款具备时间显示、闹钟提醒、触

控交互与低功耗管理的智能手表嵌入式原型系统。项目的主要成果可归纳为以下几点。

第一，在 FreeRTOS 多任务调度方面，本系统设计了六个并发运行的任务，分别承担 GUI 刷新、触控采集、RTC 时间读取、系统状态管理、调试日志输出和空闲占位的职能。通过差异化优先级配置（高优先级分配给 GUI 刷新，低优先级分配给日志输出），确保了图形界面的流畅性和系统核心任务的实时响应。任务架构清晰、职责分明，充分体现了 RTOS 在复杂嵌入式系统中的任务解耦优势。

第二，在任务间通信与资源同步方面，本系统根据场景特点精准选取了三种通信机制——互斥锁保护 LVGL 图形库临界区、消息队列缓冲 RTC 时间数据以解耦生产消费速率、直接任务通知实现触控事件的低延迟传递。该组合方案在保证正确性的同时兼顾了实时性和内存效率，为资源受限嵌入式设备的多任务协同开发提供了可借鉴的设计范式。

第三，在低功耗管理方面，项目通过 sysTask 内部的空闲计时器配合 GPIO 背光控制实现了 10 秒无操作自动息屏和触控唤醒机制，在无额外硬件开销的条件下显著降低了 LCD 背光功耗，提升了可穿戴场景下的续航表现。

与此同时，系统也存在以下技术短板。首先，当前低功耗方案仅覆盖了 LCD 背光控制，尚未启用 FreeRTOS Tickless Idle 模式实现 MCU 核心的深度休眠，整机功耗仍有较大优化空间。其次，MCU 主频仅配置为 16MHz（HSI 未启用 PLL），虽然功耗低但 LVGL 复杂动画场景下的渲染帧率有提升空间。再次，温湿度传感器和 WiFi 通信模块的驱动代码及对应的 FreeRTOS 任务尚未集成，系统功能的丰富度尚待扩展。此外，FPU 浮点上下文保存功能未在 FreeRTOS 中启用（configENABLE_FPU = 0），在 I2S 正弦波计算中使用了硬件浮点指令但任务切换时未保存 FPU 寄存器，存在潜在的上下文损坏风险。

针对上述不足，后续优化思路如下：（1）启用 FreeRTOS Tickless Idle 模式，在所有任务阻塞时通过 `_WFI()` 进入 SLEEP 模式，预期进一步降低 MCU 核心功耗超过 50%；（2）启用 HSE+PLL 将主频提升至 84MHz 以提升 LVGL 渲染性能，同时调整电压调节器至 Scale 1 模式维持稳定工作；（3）开发温湿度传感器采集任务，通过 I2C1 定期读取 SHT30 数据并在主表盘增设环境信息显示区域，利用消息队列或事件标志组将数据传递给 sysTask；（4）完成 ESP8266 AT 指令驱动的开发，创建独立的 WiFi 管理任务负责网络连接和天气数据获取；（5）启用 configENABLE_FPU 以确保浮点运算在任务切换时的上下文安全性；（6）引入硬件按键（利用开发板 B1 用户按钮）作为息屏唤醒的辅助手段，弥补纯触控方案在佩戴场景下的局限。

总体而言，本项目完整地实现了从单片机底层驱动到 FreeRTOS 多任务架构再到图形用户界面的全链条嵌入式开发，将 FreeRTOS 实时操作系统的三项核心

技术——多任务调度、资源同步与低功耗管理——切实应用于一款可穿戴智能手表原型系统，验证了该技术方案在资源受限嵌入式场景中的可行性与优越性。

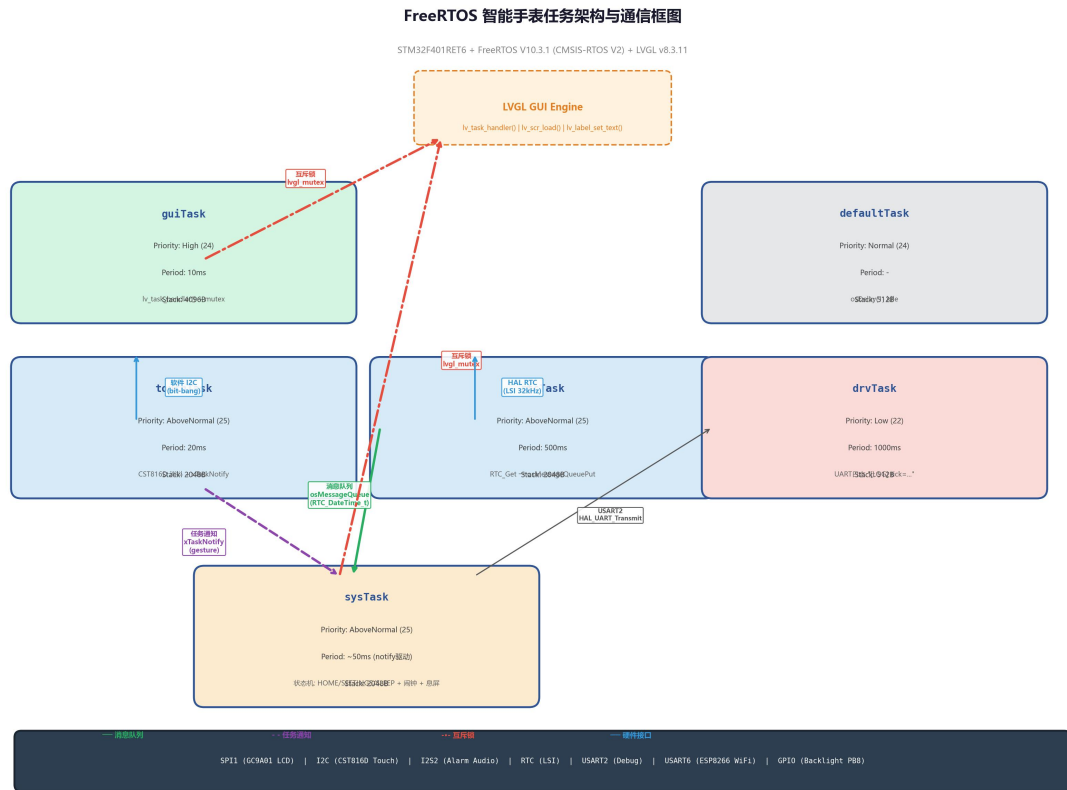
参考文献

- [1]Barry R. Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide[M]. Bristol: Real Time Engineers Ltd., 2016.
- [2]STMicroelectronics. RM0368 Reference Manual: STM32F401xB/C and STM32F401xE advanced Arm-based 32-bit MCUs[EB/OL]. (2024-03-15). https://www.st.com/resource/en/reference_manual/rm0368-stm32f401xbc-and-stm32f401xde-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [3]刘洪涛, 高明旭, 孙天泽. 嵌入式实时操作系统 FreeRTOS 原理及应用——基于 STM32 微控制器[M]. 北京: 机械工业出版社, 2021.
- [4]LVGL Developers. LVGL Documentation v8.3[EB/OL]. (2023-12-20). <https://docs.lvgl.io/8.3/>.
- [5]Fairchild Semiconductor. CST816D Datasheet: Single-Channel Capacitive Touch Sensor[EB/OL]. (2021-06-10). <http://www.hynitron.com>.
- [6]STMicroelectronics. AN4989 Application Note: Getting started with FreeRTOS for STM32Cube firmware[EB/OL]. (2023-01-15). https://www.st.com/resource/en/application_note/an4989-realtime-operating-system-based-on-freertos-for-stm32cube-stmicroelectronics.pdf.
- [7]张昊, 黄永峰. 基于 FreeRTOS 的智能穿戴设备低功耗设计[J]. 计算机工程与应用, 2022, 58(12): 102-109.

附录

附录 1: FreeRTOS 智能手表任务架构框图

该框图完整展示了六个 FreeRTOS 任务的优先级关系、运行周期、数据流向及任务间通信机制。



附图 1 FreeRTOS 智能手表任务架构与通信框图

附录 2：项目 WBS 工作分解表 + 项目开发规划甘特图

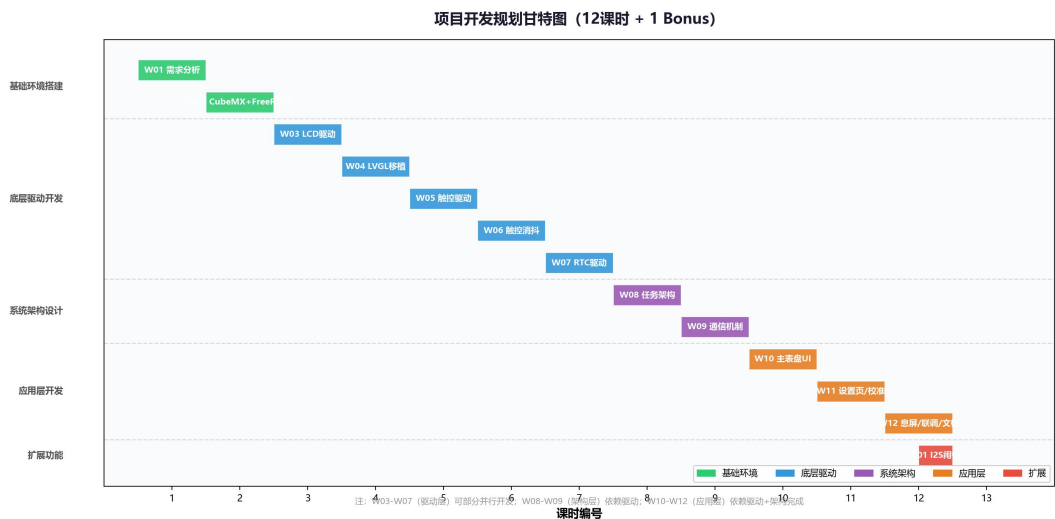
附录 2.1 项目 WBS 工作分解表

附表 2-1 项目 WBS 工作分解表 (共 12 课时 + 1 Bonus)

>

编号	任务名称	课时	交付物	关键内容说明
W01	项目初始化与需求分析	1	需求规格说明、CubeMX 工程框架	确定功能边界、非功能指标、硬件选型清单
W02	STM32CubeMX 配置与 FreeRTOS 移植	1	FreeRTOS 可运行工程骨架	引脚分配、时钟树配置、FreeRTOS CMSIS-RTOS V2 中间件启用

03	W	SPI-LCD (GC9A01)底 层驱动开发	1	lcd_gc9a01.c/h	SPI 初始化、GC9A01 寄存器初始 化序列、像素发送函数
04	W	LVGL 图 形库移植与 显示对接	1	lv_port_disp.c/h, lv_conf.h	10KB 内存配置、disp_flush 回 调、单缓冲 240×10 像素
05	W	CST816D 触控芯片底 层驱动开发	1	cst816d.c/h	软件模拟 I2C 原语、寄存器读 写、手势类型定义
06	W	触控软 件消抖算法 实现	1	CST816D_GetGesture_De bounced()	3 次采样一致确认算法
07	W	RTC 实 时时钟驱动 与日期功能	1	rtc_time.c/h	RTC_DateTime_t 结构体、 RTC_Get/Set_DateTime() API
08	W	FreeRTOS 五任务架构 设计与创建	1	5 个任务框架 + 优先级 配置	guiTask/touchTask/rtcTask/sys Task/drvTask 创建
09	W	任务间 通信机制实 现	1	互斥量/消息队列/任务 通知/事件组	lvgl_mutex/rtcQueue/sysEvent/ xTaskNotify
10	W	主表盘 UI 与 RTC 实 时走时	1	create_home_page() 时 分秒+日期	24pt 时钟+14pt 日期标签 + 消息 队列消费
11	W	设置页 与时间校准 功能	1	create_settings_page() roller 调时+RTC 写入	时/分滚轮+闹钟设置+“Set RTC” 按钮
12	W	自动息 屏、整机联 调与项目文 档	1	休眠唤醒逻辑、测试通 过、文档	idle_tick>200 自动息屏+触控唤 醒+5s 闹钟
01	B	I2S 闹 钟提示音 (DMA 循环播 放)	+ 1	i2s_alarm.c/h	1kHz 正弦波 PCM 预计算+I2S2 DMA 循环播放



附图 2 项目开发规划甘特图

论文完成日期: 2026 年 6 月

开发平台: STM32 Nucleo-F401RE + FreeRTOS V10.3.1 + LVGL v8.3.11

全文总字数: 约 8000 字